Base R Graphics — devices & par() cheatsheet

Data Visualization with R · viz-base.rsquaredacademy.com · CC BY-NC-SA 4.0

DEVICES

Devices are a stack. Opening one makes it active; closing it returns to the one underneath. Every open device must be closed or output lands somewhere you did not choose — usually `Rplots.pdf`.

`png(f, width, height)`	raster. `units` defaults to `"px"`; `res` is ignored unless `units` is physical
`pdf(f, width, height)`	vector, for print. `width`/`height` are **inches**
`svg(f, width, height)`	vector, for web. Needs a cairo build
`jpeg(f, ...)` / `tiff(...)`	raster, lossy / lossless
`dev.off()`	close the newest device; returns its name and number
`dev.cur()` / `dev.list()`	which device is active / which are open
`dev.new()`	push a copy of the current device onto the stack
`capabilities("cairo")`	`TRUE` if `svg()` will work

```
# dev.off() belongs INSIDE the loop, not after it
for (g in groups) {
  png(file.path(out, paste0(g, ".png")), width = 1800, res = 150)
  plot(x[[g]], y[[g]])
  dev.off()
}
```

PAR() — MARGINS AND TEXT

`mar` and `oma` are measured in **lines of text**, so they scale with `cex` and `pointsize`. A margin tuned on screen is wrong in a high-`res` export unless you raise `pointsize` too.

`par(mar = c(b, l, t, r))`	plot margins, in lines of text
`par(oma = c(b, l, t, r))`	outer margins, only for multi-panel figures
`par(mgp = c(3, 1, 0))`	margin lines, tick labels, axis title
`par(las = 0..3)`	tick labels: parallel, horizontal, perpendicular, vertical
`par(bty = "o", "l", "7", "u", "]", "[")`	box: `"o"`, `"l"`, `"7"`, `"u"`, `"]"`, `"["`
`par(xaxs = "i")`	`"r"` (default, 4% padding) or `"i"` (exact data range)
`par(yaxs = "i")`	same, for the y axis
`par(tck = -0.02)`	tick length; negative draws outside, `0` removes
`par(cex = 1.2)`	scale all plot text
`par(family = "mono")`	base font family
`par(font.main = 2)`	point size, main text

`las = 1` is worth setting on any chart with numeric ticks — it stops y-axis labels being drawn slanted.

PAR() — PANELS AND STATE

`par(mfrow = c(2, 2))`	panels filled by row
`par(mfcol = c(2, 2))`	panels filled by column
`layout(matrix, widths, heights)`	ragged panels; `widths`/`heights` in 0–1
`layout(1)`	cancel `layout()`, back to a single panel
`par(new = TRUE)`	draw onto the current plot without clearing it
`par(no.readonly = TRUE)`	capture **all** current settings, for restoring
`op <- par(x) / par(op)`	set and capture in one call, then restore

`par()` is device state and persists until changed. Capture and restore rather than hand-writing old values back — and in a function, wrap the restore in `on.exit()` so an error cannot leave the device mangled.

```
init <- par(no.readonly = TRUE)      # snapshot everything
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
plot(x1)
plot(x2)
par(init)                            # always restore
```

EXPORT RECIPES

`units = "in", res = 300`	the combination to use for anything printed
`pdf(onefile = FALSE)`	one file per page — safer for long batch jobs
`pdf(width = 6, height = 4)`	inches, not pixels
`pointsize = 12`	raise it on small or high-`res` exports, or labels vanish
`file.info(f)$size`	print it in a batch loop; `0` means `dev.off()` was missed

COLOURBLIND-SAFE PALETTES (R >= 4.1)

```
palette.pals()                           # every built-in palette
palette.colors(palette = "Okabe-Ito")    # 9 colours, colourblind-safe
palette.colors(palette = "Okabe-Ito", alpha = 0.5)
```

Use the hex values **as R prints them** — hand-typing them from memory is how palettes get subtly wrong. The first Okabe-Ito colour is black, so drop it with `[-1]` unless you want black as your first series.

```
# greyscale check: Rec. 601 luma, not the mean of the channels
rgb  <- t(col2rgb(pal)) / 255
luma <- rgb %*% c(0.299, 0.587, 0.114)
grey <- rgb(luma, luma, luma)
```

If the greyscale version has series you cannot tell apart, colour was doing work it should not do alone. Add a second channel: `pch`, `lty`, or direct labels.

AXIS() — DRAWING YOUR OWN

`xaxt = "n"` / `yaxt = "n"`	suppress the automatic axis **first**, or you draw two
`axis(1, at, labels)`	1 bottom · 2 left · 3 top · 4 right
`axis(1, at, labels = FALSE)`	ticks with no labels
`axis(4)`	add a second, right-hand axis — usually a different unit
`axis(..., las = 1)`	force horizontal labels, e.g. for dates
`outer = TRUE`	draw on the figure margin, not the plot region

Axis positions are plain numbers, so dates work by converting and formatting.

```
when <- as.Date("2026-01-01") + 0:6 * 30
plot(when, sales, type = "b", xaxt = "n")
axis(1, at = as.numeric(when), labels = format(when, "%b %d"), las = 1)
```

DRAWING PRIMITIVES

`plot(x, y, type=)`	`"p"` points · `"l"` line · `"b"` both · `"o"` overplot · `"h"` histogram · `"n"` axes only
`points()` / `lines()`	add to the **current** plot without clearing it
`segments(x0,y0,x1,y1)`	straight segments: error bars, caps, reference lines
`arrows(x0,y0,x1,y1)`	line with a head; args run **from ⟶ to**
`polygon(x, y, col=)`	closed filled shape; CI bands via `c(x, rev(x))`
`rect()` / `text()`	boxes and labels at coordinates
`legend(x, y, legend)`	x/y coords, or `"topleft"`, `"bottomright"`, …
`text(x, y, labels)`	labels at coordinates; `pos`, `offset`, `padj`
`mtext(text, side)`	margin text; side 1–4 as for `axis()`
`abline(a, b)`	reference line; also `v =`, `h =`

STATE YOU MUST RESTORE

Every call below changes device state that outlives the call itself. If you change it, restore it — or the next plot in the session silently inherits it.

call	leaves behind
`par(mfrow / mfcol)`	multi-panel layout; `layout(1)` cancels it
`par(mar/oma/mgp/las/bty)`	page geometry, persists until reset
`par(new = TRUE)`	a later `plot()` draws on top instead of clearing
an open device	output goes to the wrong place; `dev.off()` it
`setwd()`	inside a script, the working directory stays changed
`options()`	process-wide until changed back